Chapter 10

Dimensionality reduction: PCA & NMF

Continuing with the theme of unsupervised learning, this chapter introduces key approaches to *dimensionality reduction*.

1. Principal Component Analysis (PCA)

As usual, we have the standard setup for unsupervised learning, with a dataset:

$$\{\mathbf{x}^{(i)}\}_{i=1}^{N} \qquad \mathbf{x}^{(i)} \in \mathbb{R}^p$$

where, as discussed already for the general setting of unsupervised learning, we do **not** distinguish between input or output variables. Our aim is to extract information and structural understanding of the data in an unsupervised manner, guided by the dataset itself.

In this particular case, the dimensionality of the samples is very large, $p \gg 1$, and one might have the intuition (or prior knowledge) that many of those variables are unnecessary or redundant (e.g., correlated variables, noise, etc.), and the dataset might be better described by fewer, more informative variables. This is the case where it would be convenient to represent the data in a lower dimension.

> **Key Message:** The goal of dimensionality reduction is to find a description of the dataset that captures as much information as possible from the original data in reduced dimensions, i.e., going from the original number of dimensions p to a reduced dimension $m < p$.

Many times, dimensionality reduction is also a good step to facilitate other downstream tasks (e.g. classification or clustering), which might struggle to utilise very high-dimensional datasets. By summarising the data through a reduction of dimensions, many models become easier to train and computationally feasible, whilst losing next to nothing in terms of accuracy (or other quality measures).

The classic tool for dimensionality reduction is *principal component analysis* (PCA), which, in its basic form, is a *spectral linear method*. First, *spectral* methods are called in this way because they rely on properties of matrix decompositions closely aligned with matrix diagonalisation, a.k.a. 'spectral decomposition'. As you know, **the spectrum of a matrix is the set of its eigenvalues, with its associated eigenvectors**. These methods exploit spectral properties of matrices associated with data. Second, it is a *linear* method for dimensionality reduction because it transforms data to lower-dimensional representations using *linear projections*, as we now make clear.

1.1. Mathematical derivation of PCA. In PCA we aim for a description of every data point in terms of a *linear* combination basis functions $\mathbf{v}_j$:

$$\mathbf{x}^{(i)} = \sum_{j=1}^{p} a_j^{(i)} \mathbf{v}_j, \text{ such that } \mathbf{v}_j \in \mathbb{R}^p, \tag{10.1}$$

and we search for an orthonormal basis such that:

$$\mathbf{v}_j \cdot \mathbf{v}_k = \delta_{jk} := \begin{cases} 1 & \text{if } j = k \\ 0 & \text{otherwise} \end{cases}$$

From these definitions, it immediately follows that the coefficients in (10.1) are obtained as dot products (projections) on the basis:

$$a_j^{(i)} = \mathbf{v}_j \cdot \mathbf{x}^{(i)} = \mathbf{x}^{(i)} \cdot \mathbf{v}_j$$

One way to view dimensionality reduction is that we want to approximate $\mathbf{x}^{(i)}$ by:

$$\widehat{\mathbf{x}}_m^{(i)} = \sum_{j=1}^{m} a_j^{(i)} \mathbf{v}_j + \sum_{j=m+1}^{p} b_j \mathbf{v}_j \tag{10.2}$$

where the b_j are not dependent on i (i.e., we want to find the $b_j, \mathbf{v}_j$ such that they apply to all samples). In essence, we will describe each sample $\mathbf{x}^{(i)}$ in lower dimension through the m coefficients $a_j^{(i)}$, whereas the sum over the b_j terms in (10.2) reflects the error (common to all samples) that is associated with the elements of the basis $\{\mathbf{v}_j\}_{j=m+1}^{p}$.

We can write down the error made from this assumption:

$$\Delta\mathbf{x}^{(i)} = \mathbf{x}^{(i)} - \widehat{\mathbf{x}}_m^{(i)} = \sum_{j=m+1}^{p} [a_j^{(i)} - b_j]\mathbf{v}_j$$

We perform an optimisation for this approximation to find the best description of the data. Specifically, we minimise the associated mean squared error:

$$\begin{aligned} \text{MSE} &= \frac{1}{N} \sum_{i=1}^{N} \|\Delta\mathbf{x}^{(i)}\|^2 \\ &= \frac{1}{N} \sum_{i=1}^{N} \left[\sum_{j,k=m+1}^{p} (a_j^{(i)} - b_j)(a_k^{(i)} - b_k)\mathbf{v}_j \cdot \mathbf{v}_k \right] \\ &= \frac{1}{N} \sum_{i=1}^{N} \sum_{j=m+1}^{p} (a_j^{(i)} - b_j)^2 \end{aligned} \tag{10.3}$$

As we are looking to optimise both b_j and $\mathbf{v}_j$, we first start with the former and compute the following:

$$\frac{\partial \text{MSE}}{\partial b_j} = -\frac{1}{N}\sum_{i=1}^{N} 2(a_j^{(i)} - b_j)$$

At the optimum, we have:

$$\frac{\partial \text{MSE}}{\partial b_j}\Big|_{b_j} = 0 \quad \implies \quad b_j = \frac{1}{N}\sum_{i=1}^{N} a_j^{(i)}$$

Plugging the above expression into (10.3), we get:

$$\begin{aligned}
\text{MSE} &= \sum_{j=m+1}^{p} \frac{1}{N}\sum_{i=1}^{N} \Big(\underbrace{\mathbf{x}^{(i)} \cdot \mathbf{v}_j}_{a_j^{(i)}} - \underbrace{\frac{1}{N}\sum_{i=1}^{N} \mathbf{x}^{(i)} \cdot \mathbf{v}_j}_{b_j} \Big)^2 \\
&= \sum_{j=m+1}^{p} \frac{1}{N}\sum_{i=1}^{N} \left[\left(\mathbf{x}^{(i)} - \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}^{(i)} \right) \cdot \mathbf{v}_j \right]^2 \\
&= \sum_{j=m+1}^{p} \mathbf{v}_j^T \Bigg(\underbrace{\frac{1}{N}\sum_{i=1}^{N} \left[\left(\mathbf{x}^{(i)} - \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}^{(i)} \right) \left(\mathbf{x}^{(i)} - \frac{1}{N}\sum - -^{N} \mathbf{x}^{(i)} \right)^T \right]}_{\clubsuit} \Bigg) \mathbf{v}_j
\end{aligned}$$

Note that the 'clubsuit' block can be rewritten as follows:

$$\clubsuit = \Big\langle \left(\mathbf{x} - \langle \mathbf{x} \rangle \right) \left(\mathbf{x} - \langle \mathbf{x} \rangle \right)^T \Big\rangle \quad \text{with} \quad \langle \mathbf{x} \rangle = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}^{(i)},$$

which is the covariance matrix $(C_{\mathbf{x}})_{p\times p}$ of the data features. Hence the MSE can be written compactly as:

$$\text{MSE} = \sum_{j=m+1}^{p} \mathbf{v}_j^T C_{\mathbf{x}}\, \mathbf{v}_j \tag{10.4}$$

To find the *orthonormal* $\mathbf{v}_j$, we need to carry out a constrained optimisation. As you know by now, we do this using Lagrange multipliers to enforce equality constraints. Let us write down the Lagrangian:

$$\mathcal{L} = \sum_{j=m+1}^{p} \mathbf{v}_j^T C_{\mathbf{x}} \mathbf{v}_j + \sum_{j=m+1}^{p} \lambda_j (1 - \mathbf{v}_j^T \mathbf{v}_j),$$

where the λ_j are Lagrange multipliers. In order to optimise this, we follow the same approach as before:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_j} = 2\mathbf{v}_j^T C_{\mathbf{x}} - 2\lambda_j \mathbf{v}_j^T$$

where the the derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{v}_j}$ is meant as the row vector $\frac{\partial \mathcal{L}}{\partial \mathbf{v}_j} = \left(\frac{\partial \mathcal{L}}{\partial v_j^1}, ..., \frac{\partial \mathcal{L}}{\partial v_j^p} \right)$. The solution is then given as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_j} = 0 \quad \implies \quad C_{\mathbf{x}} \mathbf{v}_j = \lambda_j \mathbf{v}_j \tag{10.5}$$

This solution shows that the Lagrange multipliers λ_j are simply the eigenvalues of $C_{\mathbf{x}}$ with corresponding eigenvectors $\mathbf{v}_j$: **the optimal basis to expand is the basis of the**

eigenvectors of $C_{\mathbf{x}}$. In other words, if we expand the data in this basis, we will be optimally reducing the MSE defined previously. Its optimal value is given by:

$$\text{MSE} = \sum_{j=m+1}^{p} \mathbf{v}_j{}^T C_{\mathbf{x}} \mathbf{v}_j = \sum_{j=m+1}^{p} \mathbf{v}_j{}^T \lambda_j \mathbf{v}_j = \sum_{j=m+1}^{p} \lambda_j$$

that is, the MSE of the lower-dimensional approximation is the sum of the eigenvalues that are discarded when reducing dimensions from p to m.

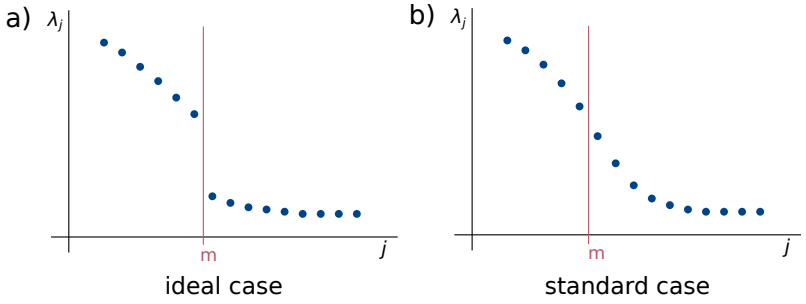


Figure 10.1. Finding an optimal m for dimensionality reduction is not always easy. In a), we can see an ideal case scenario, where some eigenvalues are large, but at some point there is a drastic drop, leading to the remaining eigenvalues to be quite small. In this case, deciding on a good m is natural. However, the usual case is seen in b), where the eigenvalues are on a much more steady decline and one must fix m according to some tolerance ε.

Finding an optimal m is then a decision made based on the eigenvalues. This can be visualised by plotting the eigenvalues, ordered according to decreasing value, as can be seen in Figure 10.1. In general, one must decide on a tolerance ε and find an m, such that $\sum_{j=m+1}^{p} \lambda_j < \varepsilon$, which will keep the error within that tolerance.

From (10.5), the total variance of the data V can be calculated as $V = \sum_{j=1}^{p} \lambda_j$, hence the fraction of variance explained by a certain component j (also called 'explained variance ratio') is given by λ_j / V. Ideally, one would choose the number of components to include by adding the **explained variance ratio** of each component until one reaches a total of **around 80%**.

Key Message: In PCA, having chosen the number of dimensions m, the approximated data in the lower, m-dimensional representation are given by:

$$\widehat{\mathbf{x}}^{(i)} = \sum_{j=1}^{m} \left(\mathbf{x}^{(i)} \cdot \mathbf{v}_j \right) \mathbf{v}_j \quad \text{where} \quad C_{\mathbf{x}} \mathbf{v}_j = \lambda_j \mathbf{v}_j$$

i.e., they are given by an expansion in the eigenvectors $\mathbf{v}_j$ of the data covariance matrix $C_{\mathbf{x}}$. These are called the *principal components*, thus the name of the method.

In Figure 10.2, we illustrate visually the difference between linear regression and PCA. Whilst in linear regression, the aim is to find a hyperplane that minimises the mean squared error of prediction (see Chapter 1), in PCA the view shifts towards errors normal to the hyperplane, i.e., the goal is to find the hyperplane that minimises the sum of distances normal to the plane. Hence, the hyperplanes obtained through each method are *not the same*. The intuition of why this happens is clear: standard linear regression is a supervised learning method where we divide our variables into inputs and outputs (i.e., independent and dependent variables). The error is then assigned to the output

observations (dependent variables), with no error in the inputs (hence the definition of the MSE in that case). In the case of PCA, there is no separation between input-output variables (as it is an unsupervised method). Hence the error is assigned to *all* variables, and the MSE depends on the errors normal to the hyperplane spanned by the principal components because PCA performs an orthogonal projection onto them.

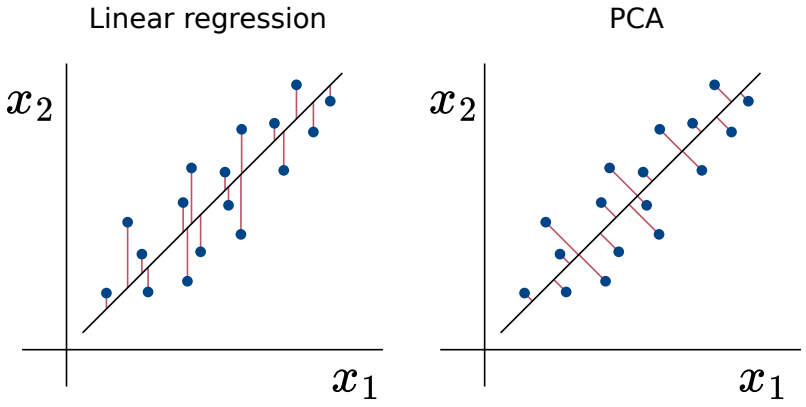


Figure 10.2. Geometric picture of PCA as opposed to linear regression.

Quick Aside: connection of PCA to Singular Value Decomposition. Singular Value Decomposition (SVD) is a standard matrix factorisation method, analogue to the diagonalisation of square matrices, but for rectangular matrices. In short, if we have a rectangular matrix $\mathbf{A}$ of dimensions $N \times p$ (let us take here $N > p$) and of rank R, the SVD decomposition of $\mathbf{A}$ has the form:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

where $\mathbf{U} = \big(\mathbf{u}_1, \ldots, \mathbf{u}_N\big)$ is a $N \times N$ matrix of the left eigenvectors $\mathbf{u}_j$, $\mathbf{V} = \big(\mathbf{v}_1, \ldots, \mathbf{v}_p\big)$ is a $p \times p$ matrix of the right eigenvectors $\mathbf{v}_j$, the matrix $\mathbf{\Sigma}$ is a $N \times p$ matrix containing non-zero values $\sigma_1, \ldots, \sigma_R$ only along the diagonal - the *singular values*. They are the square root of the eigenvalues of $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$:

$$\left\{ \begin{array}{l} (\mathbf{A}^T\mathbf{A})\mathbf{v}_k = \sigma_k^2 \mathbf{v}_k \\ (\mathbf{A}\mathbf{A}^T)\mathbf{u}_k = \sigma_k^2 \mathbf{u}_k \end{array} \right\}$$

meaning that left and right eigenvectors (or singular vectors) come in pairs with the *same* σ_k^2. A rank m approximation of $\mathbf{A}$ is then given by:

$$\mathbf{A} \approx \widehat{\mathbf{A}}_m = \sum_{j=1}^{m} \sigma_j \, \mathbf{u}_j \mathbf{v}_j^T =: \mathbf{U}_m \mathbf{\Sigma}_m \mathbf{V}_m^T$$

where $\mathbf{U}_m$ and $\mathbf{V}_m$ are truncated matrices with only the first m vectors $\mathbf{u}_j$ and $\mathbf{v}_j$, respectively. For more details, we refer to Deisenroth, Faisal, Ong *Mathematics for Machine Learning*, section 4.5.

If the matrix $\mathbf{A}$ is the dataset matrix $\mathbf{X}$:

$$\mathbf{X}_{N\times p} = \begin{pmatrix} x_1^{(1)} & \ldots & x_p^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(N)} & \ldots & x_p^{(N)} \end{pmatrix} = \begin{pmatrix} {\mathbf{x}^{(1)}}^T \\ \vdots \\ {\mathbf{x}^{(N)}}^T \end{pmatrix}$$

then, we can apply SVD to effectively reduce the dimensions of our data. Let us look at the matrix of *centred data*, obtained as:

$$\mathcal{X}_{N\times p} := \mathbf{X} - \frac{1}{N}\mathbf{1}\mathbf{1}^T\mathbf{X} = \underbrace{\left(I - \frac{1}{N}\mathbf{1}\mathbf{1}^T\right)}_{centring\ matrix}\mathbf{X},$$

In the above, we have introduced column vector of ones $\mathbf{1}_{N\times 1}$ such that:

$$\frac{1}{N}\mathbf{1}\mathbf{1}^T\mathbf{X} = \begin{pmatrix} \langle x_1\rangle & \dots & \langle x_p\rangle \\ \vdots & \ddots & \vdots \\ \langle x_1\rangle & \dots & \langle x_p\rangle \end{pmatrix}_{N\times p}$$

having denoted $\langle x_j\rangle = \frac{1}{N}\sum_{i=1}^{N} x_j^{(i)}$. In this way:

$$(\mathcal{X}^T\mathcal{X})_{p\times p} = N\,C_{\mathbf{x}} \qquad (C_{\mathbf{x}})_{p\times p} : \text{Data covariance matrix} \tag{10.6}$$

Building upon these results, we see that PCA follows from the SVD of $\mathcal{X}$:

$$\mathcal{X}_{N\times p} = \mathbf{U}_{N\times p}\mathbf{\Sigma}_{p\times p}\mathbf{V}^T_{p\times p}$$

and a rank m (with $m < p$) approximation of $\mathcal{X}$, $\widehat{\mathcal{X}}_{N\times p}$, can be found by:

$$\widehat{\mathcal{X}}_{N\times p} = \mathbf{U}_m\mathbf{\Sigma}_m\mathbf{V}^T_m =: \mathbf{X}_{\text{PCA}}\mathbf{V}^T_m$$

where the $N \times m$ matrix:

$$\mathbf{X}_{\text{PCA}} = \mathbf{U}_m\mathbf{\Sigma}_m \tag{10.7}$$

expresses the approximation $\widehat{\mathcal{X}}$ in terms of the m vectors contained in $\mathbf{V}^T_m$, which are the eigenvectors of the covariance matrix (see (1.1) and (10.6)). (Note that $\mathbf{\Sigma}$ contains the *ordered* singular values.) The matrix $\mathbf{X}_{\text{PCA}}$ thus contains the coordinates of each of the N data points in the m-dimensional space of the principal components (what we called $a_j^{(i)}$ in (10.2), with $j = 1, \dots, m$ and $i = 1, \dots, N$).

2. Extensions of PCA

The basic ideas of dimensionality reduction that have been introduced through PCA can be extended in many directions. We now briefly introduce two of them:

- how to deal with nonlinear data;
- how to produce sparse representations.

2.1. Nonlinear extensions: Kernel PCA. PCA is a linear method, i.e., it finds linear projections onto a space of reduced dimensionality. This begs the question of how to deal with nonlinear data that elude such methods, e.g., see Figure 10.3a. A first extension of PCA introduces the concept of kernels to dimensionality reduction. This should come as no surprise, since we have already introduced kernel functions as nonlinear extensions of SVMs in Chapter 6. The idea here is similar: to obtain a non-linear version of PCA by applying the kernel trick, substituting dot products in the mathematical formulation of the PCA algorithm by kernel functions.

The basic intuition behind the applicability of kernelisation is that PCA relies on the eigenvalues of $(\mathcal{X}^T\mathcal{X})_{p\times p}$, and/or $(\mathcal{X}\mathcal{X}^T)_{N\times N}$ (see aside above). The latter is precisely what can be thought of as 'kernel' matrices in this context. Indeed, the elements of the matrix $\mathcal{X}\mathcal{X}^T$ correspond to dot products of the vectors describing the *centred* p-dimensional samples:

$$\left(\mathcal{X}\mathcal{X}^T\right)_{ij} = \mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)} \quad \text{Assuming: } \mathbf{x}^{(i)} \leftarrow (\mathbf{x}^{(i)} - \langle\mathbf{x}\rangle) \tag{10.8}$$

i.e., that the original variables $\mathbf{x}^{(i)}$ are already centred. Instead of these dot products, we can now consider a kernel function $k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, as properly defined in Chapter 6.

More precisely, like in kernel SMVs, the main idea in kernel PCA is to introduce a non-linear mapping $\boldsymbol{\phi}(\mathbf{x})$ that lifts the data to D dimensions, with $D > p$ (potentially $D >> p$). Let us assume that the transformed data vector $\boldsymbol{\phi}(\mathbf{x})$ is already centred, i.e., $\sum_{i=1}^{N} \boldsymbol{\phi}(\mathbf{x}^{(i)}) = 0$. We define the $N \times N$ matrix $\mathbf{K}$ (also called the Gram matrix) as:

$$K_{ij} = \boldsymbol{\phi}(\mathbf{x}^{(i)}) \cdot \boldsymbol{\phi}(\mathbf{x}^{(j)}) \equiv k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \tag{10.9}$$

While in PCA we are interested in the spectral decomposition of $(C_{\mathbf{x}})_{p\times p}$, in kernel PCA we look at the one of:

$$(C_{\boldsymbol{\phi}})_{D\times D} = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{\phi}(\mathbf{x}^{(i)})\boldsymbol{\phi}(\mathbf{x}^{(i)})^T \tag{10.10}$$

which has eigenvalues λ_j and eigenvectors $\mathbf{v}_j$:

$$C_{\boldsymbol{\phi}}\mathbf{v}_j = \lambda_j \mathbf{v}_j \qquad j = 1, .., D \tag{10.11}$$

The eigenvectors $\mathbf{v}_j$ play the role of *non-linear* principal components. They can be expressed as a function of the set of $\boldsymbol{\phi}(\mathbf{x}^{(i)})$ through a set of coefficients $a_j^{(i)}$:

$$\mathbf{v}_j = \sum_{i=1}^{N} a_j^{(i)} \boldsymbol{\phi}(\mathbf{x}^{(i)}) \qquad j = 1, .., D \tag{10.12}$$

As we saw for SVMs (chapter 6), the introduction of kernels is convenient because we can work directly with them, without having to work out explicitly $\boldsymbol{\phi}(\mathbf{x})$. This is the key point of the kernel trick, and is valid also in this setting.

Kernel PCA construction: By plugging (10.12) into (10.11) and using the kernel definition (10.9), one obtains:

$$\mathbf{K}\mathbf{a}_j = N\lambda_j \mathbf{a}_j \quad \text{where: } \mathbf{a}_j = \{a_j^{(i)}\}_{i=1}^{N} \tag{10.13}$$

i.e., the coefficients $a_j^{(i)}$ in (10.12) are found as the eigenvectors of the Gram matrix $\mathbf{K}$. As a result, for a given data point $\mathbf{x}$ non-linearly mapped into $\boldsymbol{\phi}(\mathbf{x})$, its projection onto the jth non-linear principal component is given by:

$$\boldsymbol{\phi}(\mathbf{x}) \cdot \mathbf{v}_j = \sum_{i=1}^{N} a_j^{(i)} \boldsymbol{\phi}(\mathbf{x}) \cdot \boldsymbol{\phi}(\mathbf{x}^{(i)}) = \sum_{i=1}^{N} a_j^{(i)} k(\mathbf{x}, \mathbf{x}^{(i)}) \tag{10.14}$$

i.e., it can also be cast in terms of the kernel function only.

The result of kernel PCA can be seen as performing standard PCA in the non-linearly transformed space, resulting in non-linear dimensionality reduction in the original data space.

Kernel PCA in practice: centring the kernel. In equation (10.10), we have assumed $\boldsymbol{\phi}(\mathbf{x})$ to be already centred. This is not guaranteed to be the case even if $\mathbf{x}$ is centred, because $\boldsymbol{\phi}(\mathbf{x})$ lives in a different, D-dimensional space. In practice, $\boldsymbol{\phi}(\mathbf{x})$ won't be typically centred, hence one needs to work with:

$$\tilde{\boldsymbol{\phi}}(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x}) - \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{\phi}(\mathbf{x}^{(i)})$$

and the corresponding kernel $\tilde{K}_{ij} = \tilde{\boldsymbol{\phi}}(\mathbf{x}^{(i)}) \cdot \tilde{\boldsymbol{\phi}}(\mathbf{x}^{(j)})$. With some algebra, it can be shown that this centred version of the kernel can be found as:

$$\tilde{\mathbf{K}} = \mathbf{K} - \mathbf{1}_N\mathbf{K} - \mathbf{K}\mathbf{1}_N + \mathbf{1}_N\mathbf{K}\mathbf{1}_N \tag{10.15}$$

where $\mathbf{1}_N$ is a $N \times N$ matrix with all elements equal to $1/N$. As such, this centred version of the Gram matrix performs a rescaling of its elements by N along with the mean subtraction. (See section 12.3 in Bishop, *Pattern Recognition and Machine Learning* for more details on the derivation). Once $\tilde{\mathbf{K}}$ has been computed from the data, the construction of kernel PCA (expressions 10.13 and 10.14) is the same, with $\mathbf{K} \to \tilde{\mathbf{K}}$.

Normalising the eigenvectors. The normalisation condition for the eigenvectors $\mathbf{v}_j$ translates into a normalisation condition for the eigenvectors $\mathbf{a}_j$ in (10.13):

$$1 = \mathbf{v}_j \cdot \mathbf{v}_j = \mathbf{a}_j^T\mathbf{K}\mathbf{a}_j = N\lambda_j\mathbf{a}_j \cdot \mathbf{a}_j$$

that we have to enforce before using their elements for the projections (10.14). With the centred Gram matrix (10.15), which already includes a rescaling by N, the normalisation condition becomes:

$$1 = \mathbf{v}_j \cdot \mathbf{v}_j = \mathbf{a}_j^T\tilde{\mathbf{K}}\mathbf{a}_j = \lambda_j\mathbf{a}_j \cdot \mathbf{a}_j$$

λ_j being now the eigenvalues of $\tilde{\mathbf{K}}$.

Example. An example of a kernel is the radial basis (or Gaussian) kernel function:

$$K_{ij} = k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = e^{-\frac{\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2}{c}} \tag{10.16}$$

where c is a hyperparameter controlling the width of the kernel. Kernel PCA would work well with the data shown in Figure 10.3a, and Figure 10.3b shows the outcome of applying precisely this kernel PCA with a radial kernel. By effectively projecting the data onto a non-linear manifold spanned by the non-linear principal components, kernel PCA produces a representation that separates more straightforwardly data points with different properties (as indicated by the colour).

Finally, note that the standard PCA algorithm is recovered as a special case if we use the linear kernel (10.8).

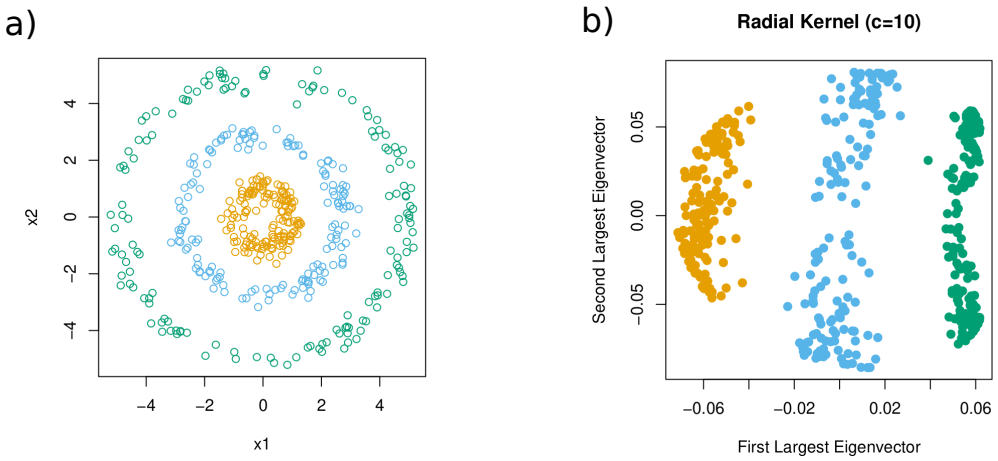


Figure 10.3. a) Example of non-linearly distributed data, that standard PCA is not able to handle very well. Instead, we need to use kernel PCA methods to obtain a more informative representation of such data. Figure adapted from Hastie, Tibshirani, Friedman, *The Elements of statistical learning*, Chap. 14.

2.2. Sparse PCA. In this second extension to PCA, *sparse PCA*, the aim is to alleviate the fact that PCA yields non-sparse descriptions of the data. In many cases this is not desirable, and so the goal of this method is to increase interpretability through sparsity. This concept is not new either—we have seen similar ideas previously when considering sparse versions of linear regression in Chapter 1.

First, recall our concise representation extracted from PCA in (10.7):

$$(\mathbf{X}_{\text{PCA}})_{N\times m} = \mathbf{U}_m\mathbf{\Sigma}_m$$

whose rows contain the coordinates of each data point in terms of the m principal vectors $\mathbf{V}_m = \big(\mathbf{v}_1 \dots \mathbf{v}_m\big)_{p\times m}$. By construction, $\mathbf{V}$ is a *non-sparse* basis in terms of the p original variables of the dataset, which in turn leads to limited interpretability.

To increase the sparsity of the representation, sparse PCA considers penalty terms that shrink the magnitude of the basis' vectors. To introduce its formulation, we need first to highlight a nice property of PCA: it can be shown that the principal components $\big(\mathbf{v}_1 \dots \mathbf{v}_p\big)$ are such that the linear combination $\mathcal{X}\mathbf{v}_1$ has the highest variance among all linear combinations of the p features; $\mathcal{X}\mathbf{v}_2$ has the highest variance among all linear combinations under the condition that $\mathbf{v}_2$ is orthogonal to $\mathbf{v}_1$, etc. In other words, principal components identify, in order, the directions of maximal variance in the data. An alternative way of deriving the original PCA is therefore by maximisation of the data variance:

$$\max_{\mathbf{v}} \mathbf{v}^T(\mathcal{X}^T\mathcal{X})\mathbf{v}, \text{ where } \mathbf{v} \in \mathbb{R}^p \text{ such that } \mathbf{v}^T\mathbf{v} = 1$$

We can induce sparsity on the vectors $\mathbf{v}_j$ by adding extra constraints, similarly to what we did in later sections of chapter 1 with LASSO:

$$\sum_{j=1}^{p} |\mathbf{v}_j| \leq t$$

This will essentially lead to PCA attempting to find a basis that tries to maximise the data variance (or, equivalently, to minimise the quadratic error) under sparsity constraints, which we can write down as:

$$\{\mathbf{v}_j^{\text{LASSO}}\}_{j=1}^{p}$$

and the $p\times m$ matrix $\mathbf{V}_m^{\text{LASSO}}$ will be sparser than the ordinary $\mathbf{V}_m$, with $\mathbf{v}_j^{\text{LASSO}}$ containing many zeros:

$$\mathbf{v}_j^{\text{LASSO}} = \begin{pmatrix} \blacksquare \\ 0 \\ \vdots \\ \blacksquare \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

The ■ stand for the non-zero elements indicating the fact that the sparse principal components can be expressed in terms of few of the original coordinates of the data. The fewer non-zero elements give a sparser representation of the main direction of variation in the data, helping identify the few features that contribute most to it. This is how sparsity in this context enhances interpretability: it performs feature selection, facilitating the generation of hypothesis based on a focussed set of relevant features. Please see Hastie, Tibshirani, Friedman, *The Elements of statistical learning*, Section 14.5.5 for the discussion of sparse PCA in real data analysis tasks.

3. Non-negative Matrix Factorisation (NMF)

Another possible limitation of PCA, i.e., the standard SVD matrix factorisation is not restricted to positive values in the matrices. However, in many applications, the data variables are restricted to be non-negative. An example of such datasets is images (where the description of the image is a grayscale value) or datasets of counts of events. Hence one would want to have a description that respects this *positivity*, and expresses the reduced representation as a linear combination of non-negative components.

The method that achieves this is called *Non-negative Matrix Factorisation* (NMF). This method was known from the 1970's and 80's but it was revitalised by Seung and Lee in a 1999 seminal paper `https://www.nature.com/articles/44565`.

To see how this is done, we once again remind ourselves of standard PCA. The approximation of our centred data, $\widehat{\mathcal{X}}$, by a projection onto the first m principal components can be written in terms of a matrix factorisation via the connection of PCA to SVD (see previous aside):

$$\widehat{\mathcal{X}} = \mathbf{U}_m \mathbf{\Sigma}_m \mathbf{V}_m^T =: \mathbf{X}_{\text{PCA}} \mathbf{V}_m^T$$

Similarly to PCA, in NMF the basic idea is to perform dimensionality reduction via matrix factorisation. We start from our usual $N \times p$ data matrix $\mathbf{X}$:

$$\mathbf{X}_{N\times p} = \begin{pmatrix} x_1^{(1)} & \dots & x_p^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(N)} & \dots & x_p^{(N)} \end{pmatrix} = \begin{pmatrix} {\mathbf{x}^{(1)}}^T \\ \vdots \\ {\mathbf{x}^{(N)}}^T \end{pmatrix} =: \begin{pmatrix} X_{11} & \dots & X_{1p} \\ \vdots & \ddots & \\ X_{N1} & & X_{Np} \end{pmatrix},$$

where we have introduced the notation X_{ij} for convenience.

> **Key Message:** In NMF, the data matrix $\mathbf{X}$ is approximated by a lower-dimensional representation given by:
>
> $$\boldsymbol{X} \approx \boldsymbol{W}\boldsymbol{H}, \quad \text{such that} \quad W_{ij} \geq 0, H_{ij} \geq 0 \quad \text{for all} \quad i, j$$
>
> i.e., where we require a factorisation into matrices with non-negative elements.

In the above, $\boldsymbol{W}$ is $N \times r$ and $\boldsymbol{H}$ is $r \times p$ matrix, where $r \ll p$ is the number of basis vectors, as follows:

$$\begin{pmatrix} X_{11} & \dots & X_{1p} \\ \vdots & \ddots & \vdots \\ X_{N1} & \dots & X_{Np} \end{pmatrix} \approx \begin{pmatrix} W_{11} & \dots & W_{1r} \\ \vdots & \ddots & \vdots \\ W_{N1} & \dots & W_{Nr} \end{pmatrix} \begin{pmatrix} H_{11} & \dots & H_{1p} \\ \vdots & \ddots & \vdots \\ H_{r1} & \dots & H_{rp} \end{pmatrix} =: \begin{pmatrix} {\mathbf{w}^{(1)}}^T \\ \vdots \\ {\mathbf{w}^{(N)}}^T \end{pmatrix} \boldsymbol{H}$$

Hence each sample is represented in terms of the r factors:

$$\mathbf{x}_{p\times 1}^{(i)} = \boldsymbol{H}^T \mathbf{w}_{r\times 1}^{(i)}, \quad i = 1, \dots, N.$$

r being the reduced dimensions of this representation. $\boldsymbol{H}$ contains the non-negative equivalent of the basis for expansion (the principal components) of PCA, and the matrix $\boldsymbol{W}$ contains the non-negative coefficients expressing the data $\mathbf{x}^{(i)}$ in the basis of vectors in $\boldsymbol{H}$, thus specifying the coordinates of the lower-dimensional NMF data representation.

3.1. NMF learning algorithms. To find $\boldsymbol{H}$ and $\boldsymbol{W}$, we need to solve an optimisation problem, where we have an objective (or cost) function to minimise that quantifies the quality of the approximation, similarly to PCA.

In the literature on NMF, there exist variants of such cost function. The two most standard cost functions are the ones analysed by Lee and Seung in `https://papers.nips.cc/paper/2000/file/f9d1152547c0bde01830b7e8bd60024c-Paper.pdf`. The first is the squared Euclidean distance between the data $\boldsymbol{X}$ and the product $\boldsymbol{WH}$:

$$||\boldsymbol{X} - \boldsymbol{WH}||^2 = \sum_{ij}(X_{ij} - (\boldsymbol{WH})_{ij})^2$$

Note that this function is not convex in both matrices together, therefore one uses techniques from numerical optimisation to find local minima. To this end, Lee and Seung introduced **multiplicative update rules** to iteratively learn a non-negative matrix factorisation. Since the joint optimisation is a very hard problem, the basic idea of the multiplicative update rules is to first fix the factor $\boldsymbol{W}$ and to optimise with regard to $\boldsymbol{H}$, and then to fix the learned factor $\boldsymbol{H}$ and to optimise with regard to $\boldsymbol{W}$. The multiplicative rules that guarantee a non-increasing Euclidean distance are:

$$W_{ij} \leftarrow W_{ij}\frac{(\mathbf{X}\mathbf{H}^T)_{ij}}{(\mathbf{W}\mathbf{H}\mathbf{H}^T)_{ij}} \tag{10.17}$$

$$H_{jk} \leftarrow H_{jk}\frac{(\mathbf{W}^T\mathbf{X})_{jk}}{(\mathbf{W}^T\mathbf{W}\mathbf{H})_{jk}} \tag{10.18}$$

The multiplicative update rules are guaranteed to converge to limiting matrices $\boldsymbol{W}$ and $\boldsymbol{H}$ forming a local minimum of the cost function. One major benefit of using multiplicative update rules is that the non-negativity constraints are enforced automatically. Furthermore, the multiplicative update rules are easy to implement and the convergence is relatively fast (see coding task).

The second type of cost function is a divergence defined as:

$$D(\boldsymbol{X}||\boldsymbol{WH}) = \sum_{ij}\left[X_{ij}\log\left(\frac{X_{ij}}{(\boldsymbol{WH})_{ij}}\right) - X_{ij} + (\boldsymbol{WH})_{ij}\right]$$

Using the multiplicative update rules above, one obtains that the divergence reaches a plateau, however the update rules that make it strictly non-increasing are slightly different:

$$W_{ij} \leftarrow W_{ij}\frac{\sum_k H_{jk}X_{ik}/(\boldsymbol{WH})_{ik}}{\sum_k H_{jk}} \tag{10.19}$$

$$H_{jk} \leftarrow H_{jk}\frac{\sum_i W_{ij}X_{ik}/(\boldsymbol{WH})_{ik}}{\sum_i W_{ij}} \tag{10.20}$$

They provide an alternative algorithm to learn the NMF of a dataset of interest, and they are as well explored in the coding task.

> *Final Aside.* For more information about NMF, you can read the original paper by Seung and Lee: `https://www.nature.com/articles/44565` that we make available as additional reading. It illustrates nicely how learning non-negative factorisations can lead to more interpretable reduced representations in the case of images. The non-negative vectors correspond to parts of images that are more recognisable and interpretable to the human eye. You will gain some more intuition of the differences between NMF and PCA through your coding tasks.